

UNIVERSIDAD AUTÓNOMA DE CHIAPAS

Dana Michelle Constantino Méndez

Ángel Omar Hernández Ramírez

Tópicos Avanzados de Base de Datos

Documentación de propuestas de proyecto “Sistema distribuido de despacho y boletaje para transporte (Autobuses)”

5° “B”

Licenciatura en Ingeniería en Desarrollo de Tecnologías Software

Introducción y Planteamiento del Problema

La gestión de boletaje en líneas de transporte regional en el corredor costero y central del estado enfrenta el desafío de la concurrencia masiva y la disponibilidad de red. Múltiples taquillas intentan asignar los mismos asientos simultáneamente, lo que en sistemas tradicionales provoca sobreventas o bloqueos.

La regla de negocio exige permitir el ascenso y descenso de pasajeros en paradas intermedias permitiendo la fluidez de disponibilidad de asientos. Este proyecto resuelve la problemática mediante una arquitectura distribuida que garantiza consistencia matemática en las bases de datos, elimina colisiones de escritura a través de un enrutamiento inteligente (Con Pgpool-II) y aísla la contabilidad financiera de la operatividad del taller mecánico.

Herramientas de utilidad

- PostgreSQL versión 17
- Java (Backend) patrón MVC
- PgPool-II
- Distribución Ubuntu Linux

Arquitectura de Red e Infraestructura Distribuida (Carga Dividida)

El sistema abandona el modelo de servidor centralizado vulnerable a cortes de internet, implementando una topología de dos nodos principales y un middleware de gestión de tráfico:

- **Nodo 1 (Servidor Sur - Tapachula):** Nodo primario activo.
- **Nodo 2 (Servidor Centro - Tuxtla Gutiérrez):** Nodo secundario de alta disponibilidad y lectura.
- **Middleware de Enrutamiento (Pgpool-II):** Actúa como el único punto de contacto para la aplicación en Java, implementando una Separación Estricta de Lectura y Escritura para erradicar las colisiones de concurrencia:
    - Load Balancing (Lecturas)*:* Cuando las taquillas ejecutan sentencias SELECT para visualizar el mapa de asientos o consultar horarios, Pgpool divide la carga distribuyendo las peticiones entre el Nodo 1 y el Nodo 2.
    - Escritura Centralizada (Prevención de Colisiones): Todas las sentencias INSERT (venta de boletos) son interceptadas y enviadas exclusivamente al Nodo 1. Al tener un único sitio de entrada para las ventas, PostgreSQL procesa las transacciones de forma secuencial, bloqueando automáticamente cualquier intento de doble venta sin generar colisiones de replicación.

    - Failover (Watchdog): Si el Nodo 1 pierde conexión, Pgpool promueve automáticamente al Nodo 2 como el nuevo receptor de escrituras, garantizando que el sistema nunca deje de vender.

Modelo Relacional y Estrategias de Distribución

El modelo cuenta con entidades núcleo como Ruta, Viaje_Programado, Boleto, Pasajero y Mantenimiento_Unidad, las cuales operan bajo dos protocolos de distribución:

- **Replicación Lógica:** Con la escritura centralizada en el Nodo 1, la replicación transfiere instantáneamente los nuevos registros hacia el Nodo 2. Todas las terminales intermedias operan como clientes ligeros conectados a Pgpool, asegurando que todos vean la misma disponibilidad en tiempo real sin saturar la red con bases de datos locales innecesarias.
- **Logística mediante Foreign Data Wrapper (FDW):** El departamento de mantenimiento opera una base de datos aislada (taller_db). Cuando logística intenta asignar un autobús, el sistema invoca una vista federada vía FDW. Si la unidad se encuentra en "Mantenimiento Preventivo", la asignación de salida se bloquea a nivel de base de datos, evitando cruzar esquemas financieros con inventarios de refacciones mecánicas.

Conclusión de Viabilidad Operativa

La sinergia entre una aplicación robusta en Java y una arquitectura de PostgreSQL optimizada por Pgpool-II resuelve definitivamente la problemática logística del transporte de pasajeros. La Separación Estricta de Lectura/Escritura elimina las colisiones en transacciones concurrentes, mientras que la implementación avanzada de rangos (`int4range`) con índices GiST maximiza la rentabilidad al permitir la venta precisa de tramos intermedios. El resultado es un sistema tolerante a fallos, auditable y preparado para la alta demanda de la región.